

Расман 3D — Архитектура проекта

Тестовое задание Unity3D-разработчика

1. Обзор

Прототип 3D-аркады в стиле Расман для Android, реализованный в Unity 6.4 (6000.4.1f1) с использованием URP. Проект построен на паттерне MVVM и обеспечивает чистое разделение игровой логики, данных и представления. Геймплейный стек, UI и сервисы развязаны через интерфейсы и события.

2. Слои архитектуры

Проект разделён на три слоя — Model, ViewModel, View — плюс инфраструктурный уровень (Core, Services). Поток данных направлен снизу вверх через события, а команды передаются сверху вниз через вызовы методов ViewModel.

2.1. Model

Чистые классы C# без зависимостей от UnityEngine.GameObject и сцены. Содержат данные и бизнес-логику. Эмитят события об изменениях.

- GameModel — счёт, жизни, прогресс пеллет, состояние игры (GameState).
- PlayerModel — параметры персонажа (скорость), его «жив/мёртв».
- PlayerProgressModel + PlayerProgressData — накопленный между сессиями прогресс (общее число пеллет, флаги способностей).
- AbilityModel — способность с её стоимостью, кулдауном и текущим состоянием.

2.2. ViewModel

Посредник между Model и View. Подписывается на события Model, ретранслирует их через свои события для View. Принимает команды от View и применяет их к Model. Не наследуется от MonoBehaviour.

- GameViewModel — оркестрирует жизненный цикл игры (StartGame, PauseGame, ResumeGame), пересылает изменения счёта, жизней, пеллет.
- PlayerViewModel — обрабатывает движение и события игрока (например, OnPlayerDied).

- AbilityViewModel — управляет покупкой и активацией способностей, считает «отображаемое» количество пеллет (сохранённые + собранные за сессию), управляет кулдаунами.

2.3. View

MonoBehaviour-компоненты, которые отображают данные и принимают пользовательский ввод. Не содержат игровой логики — только конвертация событий ViewModel в визуальное состояние и наоборот.

- PlayerView — рендеринг персонажа, обработка ввода с джойстика.
- EnemyBase и наследники — поведение врагов на основе NavMeshAgent.
- CollectibleView — пеллеты в сцене, событие OnCollected.
- UI Views — HUD (счёт, жизни, пеллеты), панели (Win/Lose/Pause), миникарта, виртуальный джойстик, главное меню.

3. Точка входа и инициализация

GameBootstrap — единая точка входа сцены. Здесь создаются модели, viewmodels и происходит «ручная композиция» (DI без контейнера).

Последовательность инициализации:

- Загружается прогресс через PlayerProgressService.Load().
- Сцена сканируется на CollectibleView, чтобы узнать общее число пеллет.
- Создаются GameModel и PlayerModel с нужными параметрами.
- Создаются GameViewModel, PlayerViewModel, AbilityViewModel.
- Через метод Construct(...) viewmodels вкручиваются в соответствующие views (PlayerView, GameHUDView, AbilityPanelView, CollectibleView).
- Вызывается StartGame() — игра переходит в состояние Playing.

Такой подход исключает синглтоны для основного gameplay-стека и обеспечивает явную зависимость каждого компонента от того, что ему действительно нужно.

4. Состояния игры

Перечисление GameState: Start, Playing, Paused, Win, Lose. Смена состояния централизована в GameModel.ChangeState() и оповещает подписчиков через событие OnStateChanged.

GameHUDView — главный слушатель этого события. Реализует интерфейс IGameStateListener и оркестрирует UI: показывает/скрывает панели, останавливает Time.timeScale, управляет звуком и BGM.

5. Враги и поведение

Все враги наследуются от абстрактного класса EnemyBase, который инкапсулирует общую инфраструктуру: ссылку на NavMeshAgent, поиск игрока, базовые методы интерфейса IEnemy (StartChase, StopChase, ResetToSpawn). Конкретные классы переопределяют UpdateBehaviour() — это шаблонный метод, паттерн Template Method.

Класс врага	Поведение
DirectChaseEnemy	Прямое преследование. При попадании игрока в радиус двигается прямо к нему через NavMesh.
PatrolChaseEnemy	Патрулирование точек на сцене. При обнаружении игрока в радиусе переключается в Chase. После потери — возвращается к патрулю.
AmbushEnemy	Засада: ждёт у спавна. Если игрок подошёл слишком близко — атакует. После погони (по таймеру) возвращается на исходную позицию.

Применение принципов ООП:

- Наследование — EnemyBase → DirectChaseEnemy / PatrolChaseEnemy / AmbushEnemy.
- Полиморфизм — каждый враг переопределяет UpdateBehaviour() и GetBehaviourType().
- Инкапсуляция — внутреннее состояние (FSM-машина в AmbushEnemy, точки патруля) скрыто за публичным интерфейсом IEnemy.

6. Интерфейсы

Расположены в Core/Interfaces. Обеспечивают слабую связность между слоями.

- IEnemy — контракт для врагов: StartChase, StopChase, ResetToSpawn.
- ICollectible — контракт для собираемых объектов.
- IPlayerInput — абстракция над источником ввода (виртуальный джойстик или другой).
- IGameStateListener — для компонентов, реагирующих на смену состояния игры.

7. Сохранения

Подсистема сохранения построена через типизированную обёртку Saver<T> и фасад PlayerProgressService.

- Saver<T> — обёртка над JsonUtility и записью в Application.persistentDataPath.
- PlayerProgressService — статический фасад с методами Load(), Save(), Reset().
- Файл данных: player_progress.json в persistentDataPath.

Моменты сохранения: при активации платной способности (списание пеллет) и при завершении раунда (Win/Lose) — заработанные за сессию пеллеты добавляются к общему прогрессу.

8. Миникарта

Реализована через отдельную ортогональную камеру, которая рендерит сцену в RenderTexture. RenderTexture отображается на UI через RawImage.

Сцена делится на слои: основная игра рендерится главной камерой, а миникарте видны только специальные слои MinimapPlayer, MinimapEnemy, MinimapWall, MinimapPellet через настройку Culling Mask.

Маркеры создаются скриптом MinimapMarker во время старта объекта. Каждый маркер — это Quad с заранее подготовленным материалом (URP/Unlit с нужным цветом). Материалы назначаются через инспектор, что критично для

билда APK: динамический Shader.Find в рантайме ненадёжен, потому что Unity выбрасывает неиспользуемые шейдеры при сборке.

9. Стек технологий

- Unity 6.4 (6000.4.1f1)
- Universal Render Pipeline (URP)
- NavMesh / NavMeshAgent — поиск пути для врагов
- C# — основная логика
- Целевая платформа: Android (минимум API 24, IL2CPP, ARM64)

10. Структура проекта

Assets/Scripts/	
Core/	
Infrastructure/	Saver, Timer, SingletonBase
Interfaces/	IEnemy, ICollectible, IPlayerInput, IGameStateListener
Common/	GameState, EnemyBehaviourType, CameraFollow
Models/	GameModel, PlayerModel, AbilityModel, PlayerProgressData
ViewModels/	GameViewModel, PlayerViewModel, AbilityViewModel
Views/	
Player/	PlayerView
Enemy/	EnemyBase + DirectChase / PatrolChase / Ambush
Collectibles/	CollectibleView
UI/	HUD, Panels, Minimap, MainMenu, VirtualJoystick
Gameplay/	Maze, Spawning
Services/	PlayerProgressService
Audio/	Sound, SoundPlayer, Sounds
GameBootstrap.cs	

11. Соответствие требованиям ТЗ

- 3D-лабиринт и перемещение игрока — реализовано.
- Сбор пеллет — реализовано через CollectibleView и события.
- Три типа врагов с разными алгоритмами — DirectChase, PatrolChase, Ambush.
- Все базовые состояния игры (Start, Playing, Paused, Win, Lose) — реализованы.
- UI (счёт, жизни, пеллеты, кнопка паузы, панели результата) — реализован.
- MVVM и чистое разделение слоёв — реализовано.
- Использование интерфейсов между слоями — реализовано.
- Принципы ООП (наследование, полиморфизм, инкапсуляция) — продемонстрированы в иерархии врагов.
- Сенсорное управление (виртуальный джойстик) — реализовано.
- Бонусы: звуки и BGM, система сохранений с метапрогрессией, способности с кулдаунами, миникарта, главное меню — реализованы.

— *конец документа* —